



ARMORIQ

SoftWare Engineer - Assignment

Build a Guarded AI Agent with MCP Support | Time budget: 3 days | Stack: your choice

Overview

ArmorIQ is an AI agent security platform. We build the layer that decides what an autonomous agent is and isn't allowed to do - especially at the point where the agent reaches out to external tools. The Model Context Protocol (MCP) is the emerging standard for that tool layer, and a big part of our world.

This assignment asks you to build a miniature version of that idea: an AI agent that talks to MCP servers, and a policy layer that sits between the agent and those servers and actually enforces guardrails. We want to see how you think about the LLM tool-use loop, the MCP protocol, and the seam between "what the model wants to do" and "what the system will permit."

What You'll Build

A full-stack app with 3 parts. All three must be wired together so the dashboard's guardrails genuinely control the agent's behavior in real time.

1. AI Agent (Backend)

A backend service that hosts the agent and runs its tool-use loop.

- You can use any LLM API (Gemini, Claude, OpenAI, whatever you prefer)
- You should connect to MCP servers (tool discovery + tool execution via stdio or SSE transport)
- You must support at least 2 working MCP servers (one remote that already exists, e.g. context7, exa; one made by you)
- Your agent should do a proper tool-use loop (LLM decides to call tool → execute via MCP → feed result back)

Tool discovery must be live, not hardcoded - when you add a new MCP server, the agent should automatically pick up whatever tools it exposes.

2. Policy / Guardrails Dashboard (Frontend)

A simple web UI where an admin can create guardrail rules such as:

- Block specific tools entirely (e.g. "never allow delete_file")
- Require human approval before executing certain tools
- Input validation rules (e.g. "file paths must be under /sandbox/ only")
- Cost/token budget per conversation

Bonus: View conversation logs - see what the agent did, which tools it called, what was blocked by policy.

Rules created or toggled in the dashboard must take effect on the running agent without a restart. How you wire that up (polling, websockets, shared store, something else) is your call.

3. A Custom MCP Server

An MCP server you write yourself, that the agent connects to alongside the remote one.

- A simple MCP server exposing 4-5 tools (e.g. a mini database CRUD, or a file manager, or an API wrapper)
- It must follow the MCP spec properly (tool listing, schema, execution, error handling)

It should be plug-and-play - pointing your agent at this server should "just work" without any agent-side code changes.

Constraints

- No hardcoded tool lists anywhere - everything is discovered from the MCP servers at runtime
- The policy engine should be a separate, self-contained module - not scattered inline through the agent loop
- Dashboard changes must propagate to a running agent without restart
- Code should be cleanly split across agent logic, policy engine, and MCP transport - not a single giant file

Context:

ArmorIQ enforces agent behavior cryptographically at the intent layer. This assignment is a simpler version of that idea: a policy layer sits between an LLM and its tools, and decides what is and isn't allowed. Design with that spirit in mind - the policy engine is the heart of the system, not an afterthought.

Edge Cases to Think About

We'll ask about these in the follow-up call. You don't have to handle every one in code, but you should have a point of view.

- What happens when an MCP server crashes mid-tool-call?
- What happens when the agent tries to bypass a guardrail via prompt injection?
- What happens when two guardrail rules conflict?
- What happens when a tool needs human approval but the approver is offline?

Logistics

- **Deliverable:** GitHub repo + deployed link + 5-min recording walking through the system live
- **Stack:** your choice
- **Bonus points:** if your custom MCP server does something creative, and if you handle prompt injection attempts in guardrails

Submission

- **Subject:** {yourName} - Armoriq SWE intern assignment submission
- **To:** fuzail@armoriq.io
- **CC:** aniket@armoriq.io, arun@armoriq.io, pulkit@armoriq.io

Questions? Follow us at [@ArmoriQ](#) for updates and announcement on this role.